

CHIP - a walkthrough

CHIP

Three voices, a resonant filter, and a player routine that rewrites the registers fifty times a second. It sounds like a Commodore 64 because it does what a Commodore 64 did, not because it plays samples of one.

It is also the example with a *deadline*. Every other example in the tree draws; this one has 10.7 milliseconds to fill a buffer before the speaker runs dry, on a thread it does not own, in a function that may not allocate, may not lock and may not raise.

Source	examples/chip/ — four files, ~2,200 lines
The engine	chip.mojo, 579 lines — three oscillators, three envelopes, one filter
Arithmetic	integer, except the filter and the final sample
Register clock	50 Hz, the PAL vertical blank
Audio deadline	512 frames at 48 kHz — 10.7 ms
Also used by	the ABC player , which vendors chip.mojo verbatim

These documents

Chapter	What it covers
1. What it is imitating	The 6581, why this is not an emulator, and what actually makes the sound
2. A chip is a bank of registers	One flat block of memory, no globals, and why that matters
3. Oscillators	Phase accumulators, the waveform AND, the noise LFSR, ring mod and hard sync
4. The envelope	The period-stretching table — the single detail that matters most
5. The filter	A state-variable filter, and the stability bug that took a tune to find

6. The player routine	Fifty edits a second, and why that is where the music lives
7. Two threads, no glue	fn as an AURenderCallback, class as an NSView, and two bugs worth keeping
8. What to understand	The things that will surprise you, and the ones that will bite

The shortest possible summary

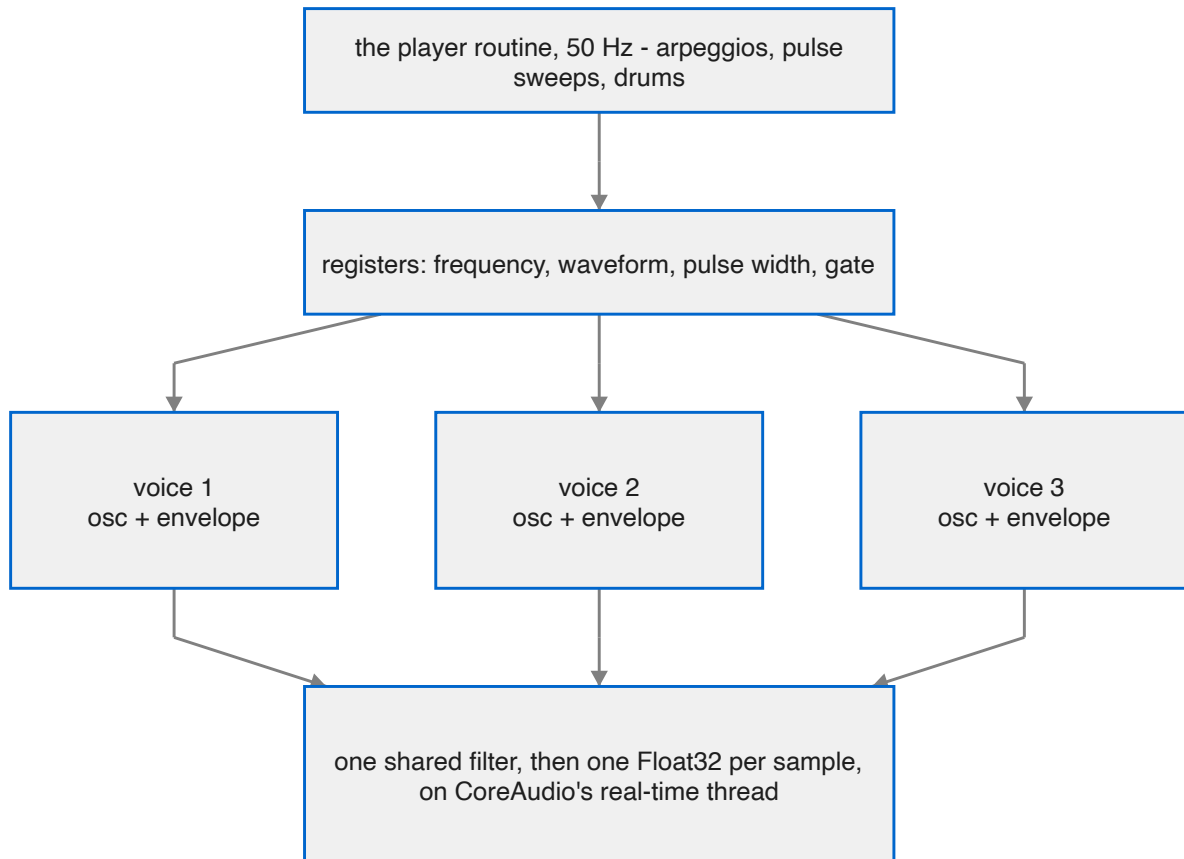
A chip-tune synthesiser is two programs, and people usually only think about the first.

The oscillator is arithmetic: a 24-bit counter that wraps, some bit manipulation to turn its value into a waveform, an envelope that scales it, and a filter. That part is a few hundred lines and it is the part everyone describes.

The player routine is the part that makes it sound like a C64. The chip has three voices and no memory. A chord is one voice switching between three notes on consecutive frames — fast enough to hear as a chord, slow enough to shimmer. A held note is kept alive by sweeping its pulse width, because a static pulse wave goes lifeless in about half a second. A drum is noise plus a downward pitch sweep, because the chip has no percussion.

So the interesting code is not the oscillator. It is this: fifty edits a second to a handful of registers, and out comes music.

And the third thing, which is why this example lives in a Mojo repository at all: the render callback CoreAudio calls is a plain Mojo `fn` installed directly into an audio unit, while the same program's window is a Mojo `class` that is a real `NSView`. One process, two threads, two ABIs, and no C or Objective-C anywhere in the build.



1. What it is imitating

The chip

In 1982 the Commodore 64 shipped with a sound chip that was, by the standards of home computers then, absurd. Most machines of the era had a square-wave beeper. This one had three oscillators with four selectable waveforms, a four-stage envelope generator per voice, a shared multimode resonant filter, ring modulation and hard sync.

It was designed by a synthesiser enthusiast who set out to build a proper subtractive synthesiser rather than a sound-effects generator, and the architecture shows it — the signal path is the one a Minimoog has, in silicon, for a home computer.

Two accidents of that design are why it has a *sound* rather than merely a specification:

- **It is part digital, part analogue.** The oscillators and envelopes are digital; the filter is analogue, and its cutoff curve varies noticeably between individual chips. Two machines do not sound quite the same.
- **The waveform selector was wired in a way its designer did not intend.** Selecting two waveforms at once ANDs their outputs together, producing timbres nobody designed, which musicians immediately began using on purpose.

The register interface is the other half of the story. There is no note concept, no polyphony manager, no sequencer. There is a bank of registers, and whatever writes them fifty times a second is the instrument.

Why this is not an emulator

The source is direct about the choice, in its first paragraph:

Not an emulator. rechip exists, it is cycle-exact, and it is thousands of lines of measured analogue behaviour. This is the other thing — the arithmetic that gives the chip its voice, written plainly, small enough to read in one sitting and fast enough to run under a real-time deadline.

That is a real distinction and worth being precise about.

A **cycle-exact emulator** reproduces the chip's behaviour sample by sample, including its documented bugs, its undocumented ones, the analogue sag of its output stage, and the fact that the filter differs between production runs. It exists so that decades of existing music plays back correctly. It is thousands of lines of measurement, and it must be, because the goal is fidelity to hardware including hardware's mistakes.

This is the **other** goal: the arithmetic that produces the character, in 579 readable lines. It cannot play back an existing chip tune bit-accurately and does not try. It can be read in a sitting, and it can be modified — which an emulator, by construction, cannot be.

Where the two diverge, the source says so rather than glossing:

The AND is faithful; what it cannot reproduce is that the real result also sags with the analogue behaviour of the output stage, so combined waveforms here are brighter than a 6581's.

And for the filter:

The 6581's cutoff curve is notoriously non-linear and differs chip to chip, so there is no correct mapping to reproduce — this is a plain linear sweep across the range the chip covers.

There is no correct answer to reproduce, so it does not pretend to have found one.

The four things that make it sound right

The header lists them, and they are worth reading as a set — each is a place where the obvious modern implementation would sound wrong:

- *the oscillators are 24-bit phase accumulators, so the pitch drifts in the same quantised way and the waveforms have the same hard edges*
- *the noise is the actual 23-bit LFSR with the actual output taps, which is why it rasps instead of hissing*
- *the envelope decays by the chip's period-stretching table rather than by an exponential curve, which is the difference between a C64 snare and a beep*
- *the pulse width is a register you are expected to modulate every frame*

Each names a substitution that a from-scratch synthesiser would naturally make, and each substitution destroys the character:

the obvious choice	what it costs
a floating-point phase, band-limited	the hard edges and quantised pitch go; it sounds clean and generic
white noise from a random generator	hisses instead of rasping; the noise loses its pitch
an exponential decay	"the difference between a C64 snare and a beep"
a static pulse width	the sound goes lifeless in about half a second

Notice that all four are the machine's *limitations*. The pitch is quantised because 24 bits was what fitted; the noise rasps because a 23-bit shift register was cheap; the envelope steps because a proper exponential would have needed a multiplier. Reproducing the sound means reproducing the constraints — which is why this is a piece of arithmetic and not a preset.

Integer, because the chip was

The chip is integer hardware, so this is integer arithmetic. The only floating point is the filter and the final sample.

Phase accumulators, waveform generation, the noise register and the envelope are all integer. The filter is float because it is the analogue part of the original; the final sample is float because that is what CoreAudio takes.

This is not a performance decision. Integer arithmetic *is* the model — a phase accumulator that wraps at 2^{24} is what produces the pitch quantisation, and a float accumulator would silently remove it.

What is in the folder

File	
chip.mojo	the synthesiser: oscillators, envelopes, filter
tune.mojo	the player routine — the part people forget
abc.mojo	a small ABC reader, so real tunes can be played
main.mojo	CoreAudio, and the window

`chip.mojo` depends only on the standard library, which is why the [ABC player](#) can vendor it as a single file:

```
# --- vendored from examples/chip/chip.mojo ---  
# abcplayer reuses the chip synthesiser's DSP engine. An IDE example is a  
# folder that opens and runs with no include flags, so the one module it needs  
# from the chip example lives here as a copy rather than as a cross-folder  
# import.
```

A verbatim copy with a header saying so, and an instruction to keep the two in step. Not elegant, and honest about why: an example folder that needs include flags to open is an example that does not open.

2. A chip is a bank of registers

There is no `Chip` struct. There is no `Voice` class. There is a `calloc`'d block of `Int64` and a set of slot numbers:

```
comptime S_TICK = 0      # samples left until the next 50 Hz frame
comptime S_CUTOFF = 1    # 11-bit filter cutoff register
comptime S_RES = 2       # 4-bit resonance
comptime S_FMODE = 3     # FILT_LP | FILT_BP | FILT_HP
comptime S_VOL = 4       # 4-bit master volume
comptime S_FRAME = 5     # frames elapsed since the chip was made
comptime S_DIRTY = 6     # filter coefficients need recomputing
```

The reason is stated as a design claim rather than a convenience:

All state lives in one flat block, because that is what a chip is: a bank of registers plus some internal counters. The block is passed to the render callback through CoreAudio's `inRefCon`, so nothing here needs a global and two chips could run at once.

Three separate benefits, and the third is the one that would be hardest to retrofit.

It matches what is being modelled

A chip is addressed, not owned. Writing `set_pulse_width(st, 1, 1100)` is poking a register, which is exactly what the machine did. There is no object graph because the hardware has no object graph.

`inRefCon` wants a pointer

CoreAudio's render callback receives one `void*` that the application chose. A flat block *is* that pointer. Nothing is boxed, nothing is looked up, and the audio thread reaches every register with one addition and one load:

```
fn vget(st: P, voice: Int, field: Int) -> Int:
    return ints(st)[unsafe_offset=V_BASE + voice * V_STRIDE + field]
```

No globals means no singleton

nothing here needs a global and two chips could run at once

This is the constraint that other examples in the tree run into repeatedly. Cocoa callbacks cannot reach locals, so state ends up in `named_globals` — and a `named_global` is a *process* global, so there is exactly one of whatever it holds.

`chip.mojo` avoids the whole problem by never having state of its own. Two chips, two blocks, no interference. The ABC player exploits this without thinking about it: it drives the same chip through a

completely different mechanism than `tune.mojo` does, and neither knows about the other.

The float slots share the memory

```
comptime S_LOW = 8      # float: filter lowpass state
comptime S_BAND = 9     # float: filter bandpass state
comptime S_F = 10       # float: filter frequency coefficient
comptime S_Q = 11       # float: filter damping
```

The float slots are the same memory read through a Float64 view; they are numbered apart so the two views never collide.

One allocation, two typed views over it:

```
fn ints(st: P) -> Pointer[Int, MutUntrackedOrigin]:
  return st.unsafe_bitcast[Int]()

fn floats(st: P) -> Pointer[Float64, MutUntrackedOrigin]:
  return st.unsafe_bitcast[Float64]()
```

Both are 8 bytes, so slot n is the same address in either view. The discipline that keeps this safe is not a type system — it is that the slot numbers are partitioned and the comment says so. Slots 0–6 are integers, 8–11 are floats, and nothing reads a slot through the wrong view.

The voice block

```
comptime V_BASE = 16
comptime V_STRIDE = 16
comptime V_ACC = 0      # phase accumulator, 24 bits with 8 fractional
comptime V_STEP = 1     # per-sample increment, same fixed point
comptime V_PW = 2       # 12-bit pulse width
comptime V_WAVE = 3     # waveform bits
comptime V_GATE = 4
...
comptime V_PREV = 15    # last accumulator, for edge detection
```

Sixteen slots per voice at a fixed stride, three voices, starting at 16. Each comment says what the register *is* on the hardware — "12-bit pulse width", "4-bit resonance" — so the model stays anchored to widths that actually existed.

`V_PREV` is the one that is not a register: it is the previous accumulator value, kept for edge detection. Hard sync and the noise shift both need to know that something *wrapped* since the last sample, which is a comparison between now and before. [Chapter 3](#) uses it twice.

Room reserved for whoever is driving

```
comptime STATE_SLOTS = V_BASE + 3 * V_STRIDE

# The player routine keeps its own state after the chip's.
comptime PLAYER_BASE = STATE_SLOTS
comptime PLAYER_SLOTS = 96
comptime TOTAL_SLOTS = PLAYER_BASE + PLAYER_SLOTS
```

The block is deliberately over-allocated. The chip uses the first 64 slots; the next 96 belong to whoever is driving it, and the chip never touches them.

That is what lets two entirely different drivers coexist. `tune.mojo` uses the player region for a score, cursors and per-voice instrument settings. The ABC player uses the *same* region for a schedule address, a sample cursor and voice assignments — and says so:

```
# Slots in the chip's player region. The chip example's own player does not
# run here -- this is a different way to drive the same chip -- so the whole
# region is free.
```

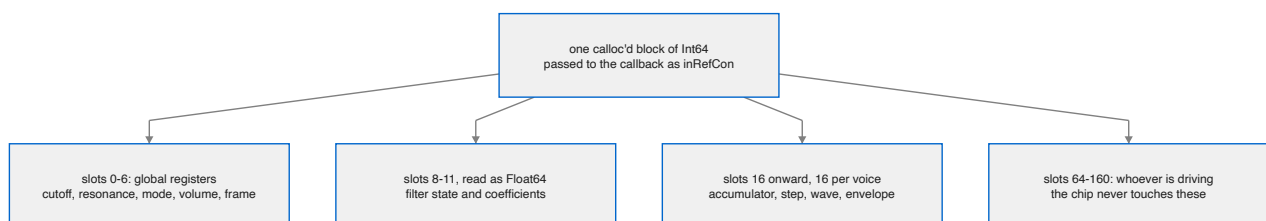
A convention rather than a mechanism, and it works because only one driver runs at a time.

The player routine, as a type

```
# The player routine, as a type: a C function pointer taking the chip. The
# alias is needed because `fn(...)` is only spelled this way in a type
# position the parser recognises as one.
comptime Tick = fn(P, /) -> None
```

The chip calls back into whatever is driving it through a plain C function pointer. It does not know what a tune is, what a note is, or what a schedule is — it knows that something wants to be called at 50 Hz and hands it the register block.

That one type is the entire interface between the synthesiser and every player built on it.



3. Oscillators

Three of them, running once per sample. Everything in this chapter happens 48,000 times a second, three times over.

The phase accumulator

An oscillator is a counter that wraps:

```
let step = vget(st, voice=v, field=V_STEP)
var prev = vget(st, voice=v, field=V_ACC)
var acc = (prev + step) & 0xFFFFFFFF
```

Add a step, mask. The wrap *is* the cycle: when the accumulator rolls over, the waveform starts again. Frequency is entirely a function of the step size.

The accumulator is 24 bits with 8 fractional bits below it — 24.8 fixed point — which is what the mask to 32 bits is doing. The extra eight bits are the difference between a pitch that is right and one that is nearly right: without them the step would be an integer number of accumulator units per sample, and most notes would land audibly off.

The frequency register is the hardware's own:

```
fn set_freq_reg(st: P, voice: Int, freq: Int):
    """The 16-bit frequency register, meaning freq * CLOCK / 2^24 Hz."""
```

The PAL machine's clock. Every frequency register in every chip tune ever written was chosen against this number.

```
comptime CLOCK_PAL = 985248
```

Keeping the register interface — rather than only accepting Hz — is what lets values from real chip tunes mean what they meant. `set_freq_hz` exists alongside it *"for tunes that were never chip data"*.

The waveform selector, and the AND

```
fn waveform(st: P, voice: Int, acc24: Int, ring_source_msb: Int) -> Int:
    """The 12-bit output of one voice's waveform selector.
```

Selecting more than one waveform ANDs them together on the real chip -- an accident of how the outputs are wired, not a design, and the source of most of the timbres people remember.

```
"""
```

```
let wave = vget(st, voice, V_WAVE)
var out = 0xFFF
```

`out` starts as all ones, and each selected waveform ANDs into it. Select none and the function returns 0; select one and you get that waveform; select two and you get their bitwise AND — which is not a mix, not an average, and not anything a synthesiser designer would choose.

The three simple ones are a few bits each:

```
if (wave & WAVE_SAW) != 0:
    out &= (acc24 >> 12) & 0xFFF

if (wave & WAVE_PULSE) != 0:
    out &= 0xFFF if ((acc24 >> 12) >= vget(st, voice, V_PW)) else 0
```

A **sawtooth** is the top 12 bits of the accumulator — the counter's own ramp, read directly. A **pulse** is a comparison against the pulse-width register: full scale above the threshold, zero below. That is the entire implementation, and it is why the pulse width is a continuous control rather than a switch.

The **triangle** folds:

```
# The triangle folds the top bit into the rest, and ring modulation
# replaces that bit with the previous voice's -- which is the whole
# of ring modulation on this chip. One XOR, and it is why bells
# and gongs sound the way they do.
var folded = acc24
if ((acc24 ^ ring_source_msb) & 0x800000) != 0:
    folded = (~acc24) & 0xFFFFF
out &= (folded >> 11) & 0xFFF
```

A triangle is a sawtooth that reverses halfway: use the top bit to decide whether to invert the rest. And **ring modulation is one XOR** in that decision — instead of this voice's top bit deciding when to fold, the previous voice's does.

That is worth pausing on. Ring modulation on a modular synthesiser is a four-quadrant multiplier. Here it is a single XOR in the fold test, because the triangle's shape is already determined by one bit. The metallic, inharmonic result — bells, gongs — comes out of that.

The noise LFSR

```
if (wave & WAVE_NOISE) != 0:
    let lfsr = vget(st, voice, V_LFSR)
    # Eight taps, scattered: bits 22, 20, 16, 13, 11, 7, 4 and 2 become
    # the output's bits 11 down to 4. The low four bits are always zero,
    # which is part of why the noise sounds coarse.
    out &= (
        ((lfsr >> 11) & 0x800)
        | ((lfsr >> 10) & 0x400)
        ...
    )
```

The noise is a 23-bit linear-feedback shift register, and the output is **not** the register — it is eight scattered bits of it, gathered into bits 11 down to 4 of a 12-bit value.

Two consequences, both audible:

- **The low four bits are always zero.** The noise is quantised to sixteen steps out of 4,096, which is part of why it is coarse.
- **The taps are not adjacent.** Consecutive samples are not smoothly related, which is what a hiss requires.

the noise is the actual 23-bit LFSR with the actual output taps, which is why it rasps instead of hissing

And the shift is not once per sample:

```
# The noise register shifts once per rising edge of accumulator
# bit 19 -- so noise pitch follows the frequency register, and a
# rising noise sweep is a rising frequency, not a filter.
if (was24 & 0x80000) == 0 and (acc24 & 0x80000) != 0:
    let lfsr = vget(st, voice=v, field=V_LFSR)
    let feedback = ((lfsr >> 22) ^ (lfsr >> 17)) & 1
    vput(st, voice=v, field=V_LFSR,
        value=((lfsr << 1) | feedback) & 0x7FFFFF)
```

Noise has a pitch. It is clocked from the oscillator, so the frequency register controls how fast the register advances — low frequency gives a slow rumble, high gives a bright hiss. This is why chip drums are written as *notes*: a snare is a noise voice at a particular frequency, and a sweep is a frequency sweep.

The feedback taps — bits 22 and 17 — are the hardware's.

Hard sync

```
# Hard sync: when the previous voice's accumulator wraps, this one
# is slammed back to zero. Two oscillators at unrelated pitches,
# one resetting the other, is the chip lead sound.
if vget(st, voice=v, field=V_SYNC) != 0:
    let src = (v + 2) % 3
    let s_now = (vget(st, voice=src, field=V_ACC) >> 8) & 0xFFFFF
    let s_was = (vget(st, voice=src, field=V_PREV) >> 8) & 0xFFFFF
    if s_now < s_was:
        acc = 0
```

The wrap is detected by comparing now against before: an accumulator that only ever increases has wrapped exactly when it appears to have gone *backwards*. That is what `V_PREV` is for.

Sync forces this oscillator to restart on the other's cycle, so it takes the other's pitch while keeping its own harmonic content — a hard, tearing timbre that sweeps as the synced voice's frequency moves.

Note `(v + 2) % 3` — the *previous* voice, wrapping. Voice 0's source is voice 2, which is the ring the hardware wires.

Order matters twice

```
# A copy: this is read before V_ACC is written below.  
var prev = vget(st, voice=v, field=V_ACC)
```

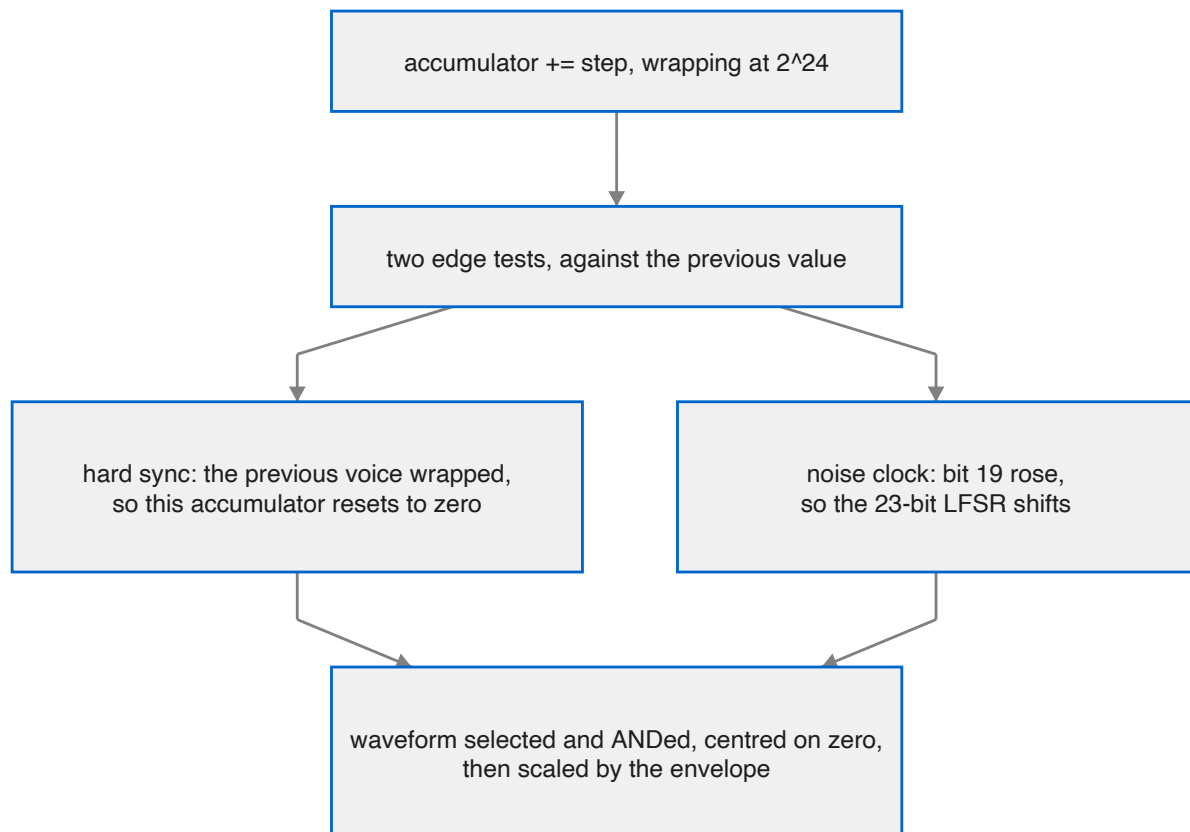
`var`, not `let`. In this dialect `let` binds to a place, so `let prev` would name the accumulator slot rather than copying it — and `prev` would read back *changed* after `V_ACC` is written a few lines later. Both the sync detector and the noise-clock edge test compare against `prev`, so both would silently stop working.

This is the trap that runs through the whole tree, and here it would have produced a synthesiser with no sync and no noise, and no error.

And the DC centring:

```
# Centre the waveform before the envelope scales it, or every  
# note-on would put a step of DC through the filter.  
let sample = Float64((w - 2048) * env) / 255.0
```

The waveform is 0...4095, unsigned. Subtracting 2048 centres it on zero *before* the envelope multiplies. The other order — envelope first, centre after — makes every note-on a step change in DC offset, which the filter turns into a thump on every note.



4. The envelope

The README makes an unusually strong claim about this one file's worth of arithmetic:

The envelope table is the single detail that matters most. Replace it with a smooth exponential and the thing stops sounding like a C64 and starts sounding like a synthesiser.

What a textbook envelope does

Attack rises to full, decay falls to the sustain level, sustain holds, release falls to silence. Every synthesiser has this, and the decay is normally exponential — a fixed *fraction* removed per unit time, which gives the smooth asymptotic curve of a plucked string.

What this one does

```
"""One sample of the envelope. Returns the level, 0..255.
```

```
The decay and release are not exponential curves. The chip counts down at
a rate that is divided further as the level falls -- once below 93, then
54, 26, 14 and 6 -- so the tail flattens in five visible steps. Replacing
that with a smooth exponential is the single change that makes this
sound like a synthesiser instead of a games machine.
"""
```

```
let level = env >> 16
var divisor = 1
if level <= 6:
    divisor = 30
elif level <= 14:
    divisor = 16
elif level <= 26:
    divisor = 8
elif level <= 54:
    divisor = 4
elif level <= 93:
    divisor = 2
let step = vget(st, voice, V_DINC if phase == ENV_DECAY else V_RINC)
env -= step // divisor
```

The rate is **piecewise constant**. Above level 93 the envelope falls at full speed; below 93 at half; below 54 at a quarter; below 26, 14 and 6 slower again, down to a thirtieth.

The hardware reason is economy. A true exponential needs a multiply per sample, and this chip had no multiplier. A counter whose *reload period* is stretched at a few thresholds is a handful of comparators, and it approximates an exponential well enough — which is to say, not very, and audibly.

The result is a decay that falls quickly, then *sticks* — five discrete flattenings on the way down. That is the shape of a C64 note, and it is why a snare made this way cracks and then hangs rather than simply fading.

The attack is linear, and only the falling phases stretch

```
if phase == ENV_ATTACK:
    # The attack is linear. Only the falling phases are stretched.
    env += vget(st, voice, V_AINC)
```

Asymmetric, matching the hardware. It also matters musically: a linear attack has a hard front edge, which is where the *snap* on a chip bass comes from.

The rate table, and the 3× rule

```
# The chip's attack times in milliseconds, 0..15. Decay and release run three
# times slower for the same index, which is why a C64 bass can have a snap on
# the front and still ring for half a second.
fn attack_ms(index: Int) -> Int:
    if index == 0: return 2
    if index == 1: return 8
    ...
    if index == 9: return 250
    if index == 10: return 500
    ...
    return 8000
```

Sixteen values from **2 ms to 8 seconds**, and the spacing is not even — roughly linear at the short end, then jumping. Index 8 is 100 ms, index 9 is 250 ms; that step is where a musician's choices actually live, and it is the hardware's.

Then:

```
vput(st, voice, V_AINC, rate_increment(attack_ms(a)))
vput(st, voice, V_DINC, rate_increment(attack_ms(d) * 3))
vput(st, voice, V_SUS, (s & 15) * 17) # 4 bits scaled to 0..255
vput(st, voice, V_RINC, rate_increment(attack_ms(r) * 3))
```

One table, three uses. Decay and release take the same index times three, which is the hardware's ratio — so the same nibble means a much longer fall than rise. The comment explains the musical consequence rather than the mechanism: *a snap on the front and still ring for half a second*.

* 17 for the sustain is `0xFF / 0xF` — a 4-bit register scaled to 8 bits so that 15 maps to 255 exactly, not 240.

The guard that stops a voice going silent forever

```
fn rate_increment(ms: Int) -> Int:
  """Envelope steps per sample, 16.16 fixed point, for a full 0..255 sweep.

  Clamped to at least one so the shortest attack still moves; a zero here
  would hang the envelope in its attack phase and the voice would never
  sound.
  """
  let samples = (ms * SAMPLE_RATE) // 1000
  if samples < 1:
    return 255 << 16
  let inc = (255 << 16) // samples
  return 1 if inc < 1 else inc
```

Two clamps for two different failures.

`samples < 1` handles a rate so fast it rounds to no samples at all — jump straight to full. And `inc < 1` handles the opposite: an 8-second release divided across 384,000 samples could round to **zero** increment, at which point the envelope never moves and the voice is stuck in its attack phase, silent, forever.

Integer division rounding to zero is the whole bug, and one comparison prevents it.

Fixed point at 16.16

The envelope level is kept as 16.16 fixed point and read out as `env >> 16`. The reason is the one above: a full sweep over 384,000 samples needs a step much smaller than one unit of the 0–255 output, so the fractional bits are where the slow envelopes actually live.

Why the shape matters more than the timing

An exponential decay and this stepped one can be tuned to the same *duration* — same time from full to silence. They still sound different, because the ear follows the shape of the fall rather than its endpoint. The stepped version lingers at low levels far longer than an exponential would, which is exactly the "hang" that makes a chip drum sound like one.

That is what the README means by *the single detail that matters most*: it is not an accuracy detail. Get the oscillators right and the envelope wrong and it sounds like a modern synthesiser playing chip waveforms.

5. The filter

One filter, shared by all three voices, with a per-voice routing switch. It is the analogue part of the original and the only floating-point arithmetic in the signal path — and it is where the most interesting bug in the example lived.

Six lines of filter

```
let f = fget(st, S_F)
let q = fget(st, S_Q)
var low = fget(st, S_LOW)
var band = fget(st, S_BAND)
low += f * band
let high = wet - low - q * band
band += f * high
```

A two-pole **state-variable filter** — the Chamberlin topology. Two state variables, two multiplies, and all three responses available at once:

```
let mode = get(st, S_FMODE)
var filtered = 0.0
if (mode & FILT_LP) != 0:
    filtered += low
if (mode & FILT_BP) != 0:
    filtered += band
if (mode & FILT_HP) != 0:
    filtered += high
```

Low, band and high are all computed anyway, so the mode register selects which to sum — and because it is a bit mask rather than an enum, combinations work. That matches the hardware, which also let you select more than one.

`f` controls cutoff, `q` controls damping. Higher resonance means *less* damping:

```
# Resonance 0..15 maps to damping 1.4 down to 0.1: higher resonance is
# less damping, and the filter rings.
let q = 1.4 - Float64(get(st, S_RES)) * 0.086
```

The bug: conditionally stable, unconditionally trusted

A state-variable filter is only **conditionally stable**. Roughly, `f + q < 2`; outside that the state grows without bound.

The original code clamped `f` to a flat 1.4 and left it there. The comment now records what happened:

```
# The two coefficients are NOT independent. A Chamberlin state-variable
# filter is stable only while f + q < 2, so the usable cutoff depends on
```

```
# the resonance chosen with it. Clamping f to a flat 1.4 was enough for
# the single cutoff and resonance the demo tunes used, and left the filter
# sitting just inside its own limit there -- f 1.099 against a bound of
# 1.116 -- with much of the rest of the range unstable. At cutoff 1700
# with resonance 5 the state diverges within a second, and everything
# after that is a buzz.
#
# Nothing found this until a tune began sweeping the cutoff, because
# nothing had ever changed it while a note was sounding.
```

Read that carefully, because it is a very good bug.

The demo settings sat at $f = 1.099$ against a bound of 1.116. Inside the limit by 1.5%. Every test passed, every tune played, and the filter was one small parameter change away from diverging — with no margin anyone had measured, because nobody knew there was a bound to have a margin against.

The flat clamp was the wrong shape. A constant limit on f cannot express a constraint that couples f and q . It happened to be safe at one point in a two-dimensional space.

Nothing found it for a long time, and the reason is the sharpest part: *nothing had ever changed the cutoff while a note was sounding*. Static settings, chosen inside the stable region, never explore the space. It took the ABC player's `[I:chip cutoff=...]` directive — a tune that sweeps the filter — to walk the parameter out of the region.

The fix makes the limit depend on the resonance actually in use:

```
var limit = 0.95 * (2.0 - q)
if limit > 1.4:
    limit = 1.4
if f > limit:
    f = limit
```

$2.0 - q$ is the stability bound; 0.95 is a 5% margin; the 1.4 cap keeps the old ceiling. High resonance now costs cutoff range, which is the real trade the topology imposes.

The second failure: NaN is sticky

Clamping the coefficients is not sufficient, because a filter can already be diverging when the coefficients change. So the state is clamped too — and the comment explains why an output clamp would not have helped:

```
# A state-variable filter is only CONDITIONALLY stable, and nothing
# stops a tune asking for a high cutoff and a high resonance at the
# same time. Left alone the state diverges, reaches infinity, and the
# next subtraction turns it into NaN -- which is sticky: every sample
# after it is NaN too, so the synth goes silent for good and only a
# restart brings it back. The output clamp below cannot help, because
# by then the damage is in the state rather than the sample.
#
# A real filter saturates, so this one does. The reset on NaN is what
# makes it recoverable rather than merely quieter.
```

```

if low != low or band != band:
    low = 0.0
    band = 0.0
if low > FILTER_CEILING:
    low = FILTER_CEILING
elif low < -FILTER_CEILING:
    low = -FILTER_CEILING

```

Three things worth extracting.

The failure is permanent. `inf - inf` is NaN, and NaN propagates through every subsequent operation. The state variables feed back into themselves, so one NaN poisons the filter for the rest of the session. Not a glitch — silence until restart.

Clamping the output cannot fix it, because the damage is in the *state*. This is a general lesson about feedback: validating what comes out is useless when the thing that is wrong is what gets fed back in.

`low != low` is the NaN test. NaN is the only value not equal to itself, and this idiom avoids needing an `isnan` import — which matters here, because this is an `fn` on the audio thread and cannot call anything that might raise.

And the justification for saturating rather than erroring: *a real filter saturates, so this one does*. Clipping is a physical behaviour of an analogue filter driven too hard. Reproducing it gives a sound rather than a silence.

Recomputing only when dirty

```
comptime S_DIRTY = 6          # filter coefficients need recomputing
```

```

if get(st, S_DIRTY) != 0:
    recompute_filter(st)

```

`recompute_filter` calls `_sin`, which is a nine-term series — expensive by audio-thread standards. The dirty flag means it runs when a register changes, not per sample. It is checked once at the top of `chip_render` and again after each 50 Hz player tick, which are the only two moments the registers can have changed.

The sine that could not be imported

```

fn _sin(x: Float64) -> Float64:
    """Sine by series.

    std.math.sin is a `def`, and a `def` may raise. The render callback is an
    `fn` -- non-raising, C-callable -- so it cannot call one. Nine terms is
    far more than a filter coefficient needs.
    """

```

The standard library's `sin` is a `def`, and a `def` may raise. This code path is reached from an `fn`, so it cannot call one. Nine terms of the Taylor series it is.

And inside it, a range-reduction comment that is the best single warning in the file:

```
# Range reduction in one step, not a loop. Subtracting 2*pi until the
# argument lands in range is harmless for a filter coefficient and quietly
# ruinous for vibrato: that argument grows with the frame counter, so the
# loop gets one iteration longer every fifty frames -- on the audio
# thread, where the cost eventually shows up as a dropout and never as an
# error.
```

The textbook `while x > 2*pi: x -= 2*pi` is fine when the argument is small. The vibrato in [chapter 6](#) calls `_sin(frame * 0.55)`, and `frame` counts up forever — so the loop gets one iteration longer every fifty frames. After ten minutes it is thousands of iterations, on the real-time thread.

The failure mode is the point. It does not error. It does not sound wrong. It gets gradually more expensive until a buffer is late, and then you hear a click — in a program that has been running for ten minutes, with nothing to point at. One multiply and one truncation instead:

```
let turns = x * 0.15915494309189535    # 1 / 2*pi
var t = x - 6.283185307179586 * Float64(Int(turns))
```

Constant time, whatever the argument.

What is not modelled

The 6581's cutoff curve is notoriously non-linear and differs chip to chip, so there is no correct mapping to reproduce — this is a plain linear sweep across the range the chip covers.

```
let hz = 200.0 + Float64(cutoff) * 5.8
```

200 Hz to about 12 kHz, linearly. The real chip's curve is famously inconsistent — the reason chip musicians talk about which revision a machine has. There is no single curve to be faithful to, so the code picks a defensible one and says that is what it did.

6. The player routine

This is the part people forget when they say "chip music". The chip has three voices and no memory of anything; everything that makes a C64 tune sound like a C64 tune happens up here, in a routine the raster interrupt called once per frame.

Fifty times a second

On the original machine, a routine ran off the raster interrupt at the vertical blank — 50 Hz on PAL — and rewrote registers. Everything a chip tune does that the chip cannot do itself happens in those fifty slots per second.

Here the same thing happens from inside the audio callback:

```
var countdown = get(st, S_TICK) - 1
if countdown <= 0:
    countdown = FRAME_SAMPLES
    put(st, S_FRAME, get(st, S_FRAME) + 1)
    tick(st)
    if get(st, S_DIRTY) != 0:
        recompute_filter(st)
    put(st, S_TICK, countdown)
```

A real machine got here by interrupt; the arithmetic is the same either way.

`FRAME_SAMPLES` is 960 — $48,000 \div 50$. A counter, and when it reaches zero the player routine is called. There is no timer, no thread and no interrupt; the sample counter *is* the clock, which is the same idea the [ABC player's scheduler](#) is built on.

And the routine is called **from inside the audio callback**:

tick is a plain C function pointer, so the chip never has to know what a tune is — and the player routine gets called from inside the audio callback, on the beat, exactly as a raster interrupt would have.

Which sets its contract:

The routine is an fn — a plain C function pointer — because it is called from inside the audio callback, on CoreAudio's real-time thread. It allocates nothing and cannot raise. Building a score does both, so that happens up front, in ordinary code.

The arpeggio

The most recognisable sound the machine made, and it is four lines:

```
# The arpeggio. Three notes, one per frame, and the ear hears a chord.
var note = n0
let n1 = pvget(st, v, PV_N1)
if n1 >= 0:
    let n2 = pvget(st, v, PV_N2)
    let width = 3 if n2 >= 0 else 2
    let which = frame % width
    if which == 1:
        note = n1
    elif which == 2:
        note = n2
```

Three voices, and a tune wants chords *and* a bass line *and* a drum. There is no fourth voice, so a chord is played by one voice switching between its three notes on consecutive frames.

At 50 Hz each note lasts 20 ms and the pattern repeats every 60 ms — about 16 Hz. That is the sweet spot: fast enough that the ear fuses it into a chord, slow enough that the fusion is imperfect and it *shimmers*.

fast enough that the ear hears a chord and slow enough that it shimmers — the arpeggio, and the single most recognisable sound of the machine.

This is a limitation that became a style. Nobody chose that sound; three oscillators and a 50 Hz interrupt chose it.

The pulse sweep

```
# Pulse-width modulation: a slow triangle, never reaching the ends,
# where the wave would go silent.
let pwm = piget(st, v, PI_PWM)
if pwm != 0:
    let cycle = (frame * pwm) % 4096
    let sweep = cycle if cycle < 2048 else (4096 - cycle)
    set_pulse_width(st, v, 512 + sweep)
```

a held note is kept alive by sweeping the pulse width every frame, because a static pulse wave goes lifeless in about half a second

A pulse wave with a fixed duty cycle has a fixed harmonic spectrum, and the ear stops attending to it very quickly. Moving the duty cycle moves the harmonics, and the note stays alive.

The triangle is built by folding a modulo — up for the first half of the cycle, down for the second — which is the same trick the oscillator's triangle uses, one level up.

The `512 +` offset is the important part: **never reaching the ends**. Pulse width 0 or 4095 means the comparison in the waveform generator is always true or always false, and the voice goes silent. Sweeping into either end would produce a note that drops out at the edge of every cycle.

Drums

```
# A drum is a note that falls. The chip has no percussion; every C64
# drum is noise plus a downward sweep, done from up here.
let drum = piget(st, v, PI_DRUM)
if drum != 0:
    let age = 0 if left < 0 else (24 - left)
    if age > 0:
        var fall = 1.0
        for _ in range(age):
            fall *= 0.82
        hz *= fall
```

Noise, plus frequency multiplied by 0.82 for each frame since the note began. After ten frames — a fifth of a second — the pitch is down to 13% of where it started.

This works because [noise has a pitch](#): the LFSR is clocked from the oscillator, so sweeping the frequency register sweeps the character of the noise from a bright crack to a low thud. That is a snare.

Note `instrument_drum` sets `WAVE_NOISE` with `set_adsr(st, voice, 0, 5, 0, 4)` — instant attack, sustain zero. The note is entirely attack and decay, which is what a percussive envelope is.

Vibrato, and a comment about `let`

```
# `var`, not `let`: `let` names the storage rather than copying it, so
# a value read that way reads back changed once the slot is written --
# and the drum's pitch below is derived from this counter.
var left = pvget(st, v, PV_LEFT)
pvput(st, v, PV_LEFT, left - 1)
```

The same rule as everywhere else in the tree, and here the consequence is specific: the drum's `age` is `24 - left`. With `let`, `left` would read back as the decremented value, `age` would be off by one every frame, and the drum's pitch sweep would be subtly wrong — audible only as "the drums sound a bit off".

```
# Vibrato, delayed slightly so short notes stay clean.
let vib = piget(st, v, PI_VIB)
if vib != 0:
    let cents = Float64(vib) * _sin(Float64(frame) * 0.55)
    hz *= 1.0 + cents / 1200.0
```

Depth in **cents** — hundredths of a semitone — which is the unit a musician thinks in, converted by `/ 1200.0` because there are 1,200 cents to an octave.

`_sin(frame * 0.55)` is the call that made `_sin`'s range reduction matter: `frame` counts up forever, so the argument grows without bound, and a loop-based reduction would get slower for as long as the program runs. See [chapter 5](#).

Instruments are just register settings with names

```
fn instrument_lead(st: P, voice: Int):
    set_wave(st, voice, WAVE_PULSE)
    set_adsr(st, voice, 0, 7, 11, 6)
    piput(st, voice, PI_PWM, 7)
    piput(st, voice, PI_VIB, 14)
    route_filter(st, voice, False)

fn instrument_bass(st: P, voice: Int):
    set_wave(st, voice, WAVE_SAW)
    set_adsr(st, voice, 0, 6, 8, 5)
    piput(st, voice, PI_PWM, 0)
    piput(st, voice, PI_VIB, 0)
    route_filter(st, voice, True)
```

Three voices, three jobs. These are the register settings, given names.

The lead is a pulse with a sweep and vibrato, unfiltered. The bass is a sawtooth — harmonically rich, so there is something for the filter to work on — routed *through* the filter. The drum is noise with a falling pitch.

No instrument abstraction, no patch format. An instrument is six register writes, and that is all it ever was.

The score

```
# One flat block of Ints, four per event: three notes and a duration. Three
# notes because an event may be a chord, and a chord on this chip is an
# arpeggio. A note of -1 is a rest.
```

Four integers per event, in `calloc`'d memory — because the audio thread walks it and may not allocate. Three note slots because the arpeggio needs three, and `-1` for a rest.

Durations are in **frames**, not seconds:

```
comptime EIGHTH = 10          # frames -- 50 Hz / 10 = 150 bpm in eighths
comptime QUARTER = 20
```

Which is how C64 tunes were always written. The 50 Hz interrupt is the clock, so tempo is a count of frames, and a tempo that does not divide evenly into 50 Hz simply does not exist.

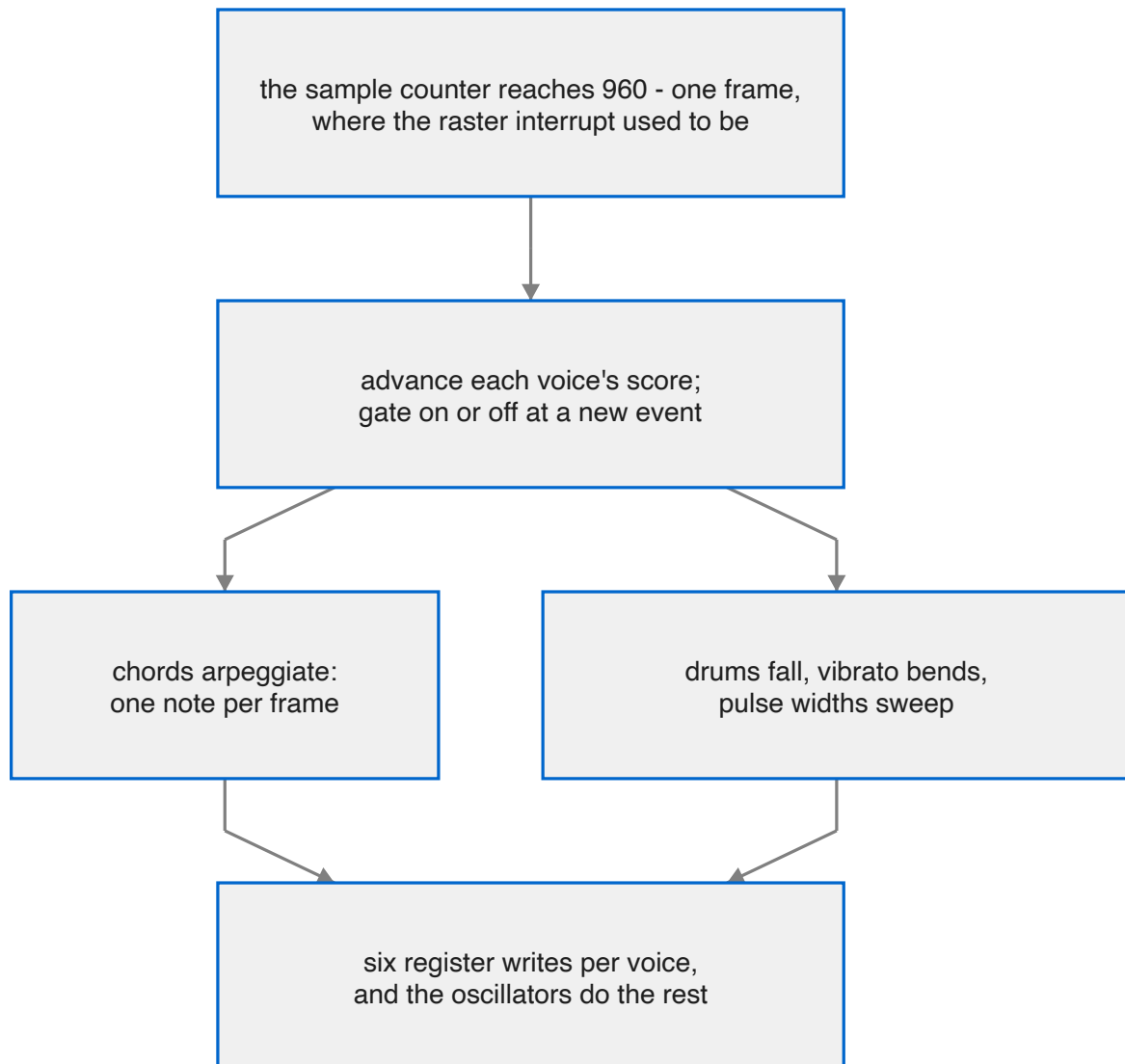
The same chip, driven two ways

The ABC player uses `chip.mojo` unchanged and does **not** use this routine:

```
fn silent_tick(st: P) -> None:  
    """The chip's own player routine, doing nothing."""  
    pass
```

Its notes come from a sample-accurate schedule rather than a 50 Hz score, so its player routine has nothing to do — but the chip still calls one, so an empty `fn` is supplied.

Two completely different drivers, one synthesiser, and the only thing they share is the `Tick` function pointer type and the register block.



7. Two threads, no glue

Every other example here draws. This one has a deadline.

That sentence is why this example exists in a Mojo repository rather than a music one.

The deadline

CoreAudio calls a render callback on a real-time thread it owns:

a C function pointer, invoked every 10.7 ms, that must fill 512 samples before the speaker runs dry. It may not allocate, may not take a lock, and may not raise. Miss the deadline and you hear it.

All three prohibitions have the same cause. The thread runs at a priority above almost everything else on the system, and anything that can make it *wait* — an allocator lock, a mutex, an exception unwinder walking tables — can make it late. Late means the buffer is unfilled, and an unfilled buffer is a click.

Not a warning. Not a log line. A click, and then the next one.

Mojo is on both sides of the boundary

This is the claim the example is built to demonstrate:

Mojo turns out to be on both sides of that boundary without a shim:

- ***fn is a foreign-callable C-ABI function.*** So render in `main.mojo` is an `AURenderCallback` — installed straight into the audio unit with `AudioUnitSetProperty`. There is no C file in this build and no Objective-C.
- ***class declares a real Objective-C class.*** So the same program's window is an `NSView` subclass that `AppKit` dispatches to normally.

One process, two threads, two languages' worth of ABI, and nothing hand-written to bridge them. That is the thing worth showing.

Consider what this normally takes. A Swift audio application still writes its render callback in C or in a carefully-annotated Swift function, because the audio unit wants a plain function pointer and Swift closures are not one. A C++ one writes the callback in C++ and the UI in Objective-C++, in two files, with a bridging header.

Here both are the same language in the same file, and the language's two function kinds *happen to be exactly the two ABIs required*:

the requirement	the language feature
a plain C function pointer that cannot raise	<code>fn</code>
an Objective-C class <code>AppKit</code> can dispatch to	<code>class</code>

`fn` being non-raising is not a convenience — it is the property CoreAudio requires, enforced by the type. You cannot accidentally install a callback that might throw, because a `def` will not convert.

And the linker cooperates:

cocoamojo links AudioToolbox for every program, the way it already links AppKit and Metal, so this builds with no extra flags.

The API that had to be the old one

(``AVAudioSourceNode`` would have been the modern API, and it takes an Objective-C block. Mojo cannot construct a block, so this uses the older AudioUnit path, where the callback is a plain function pointer. That is a real limitation, and it is also why the older API is the right one here.)

An honest limitation, stated as one. An Objective-C **block** is not a function pointer — it is a struct with an invoke pointer, a descriptor and captured variables, laid out by a compiler ABI Mojo does not implement. So the modern API is unreachable.

The older AudioUnit path takes a bare function pointer plus a `void*` context, which is precisely what this language *can* produce. The limitation and the solution are the same fact.

It is worth noting this is the same shape of constraint that shows up elsewhere in the tree: a `DeviceContext` cannot live in a `named_global`, so GPU examples drive their own event loop. Here a block cannot be constructed, so the audio example uses the C-shaped API. In both cases the older, plainer interface is the one that fits.

The context pointer

```
fn render(ref_con: P, action_flags: P, timestamp: P,
          bus: UInt32, frames: UInt32, io_data: P) -> Int32:
```

`ref_con` is the `void*` the application handed to `AudioUnitSetProperty`, and it is the chip's register block. That is why [chapter 2](#)'s flat block matters: there is nothing to unbox, no global to look up, and no synchronisation to reach the state — the argument *is* the state.

Two bugs worth keeping

The README records both, and the sentence that matters is the last one:

Both were found by measuring rather than reading: a counter that stopped advancing, then the offending value printed from the thread that could safely print it.

A let that aliased a loop counter

let drum_first = i names i's storage rather than copying it, so the recorded index followed the loop and voice 2 was pointed past the end of the score block. It read whatever the heap held; when that happened to be a note of several billion, midi_hz's octave normalisation became a loop that never finished — on the audio thread, where a hang is silence rather than a crash. It froze about three runs in five, which is exactly the failure rate that makes you blame the audio system.

Follow the chain, because every link is instructive:

1. `let` binds to a place, so `drum_first` tracked `i` instead of copying it.
2. The recorded index was therefore wrong, and voice 2 read past the score.
3. Past the score is *whatever the heap holds* — usually small, occasionally enormous.
4. `midi_hz` normalises octaves with a `while` loop, so an enormous note is an effectively infinite loop.
5. That loop runs on the audio thread, where a hang produces **silence**, not a crash and not a stack trace.
6. It happened about three runs in five.

That failure rate is the trap. A deterministic hang gets debugged. An intermittent one that produces silence gets blamed on CoreAudio, on the audio device, on the sample rate — on anything but the score index.

The fix was two things, and the second is the general lesson:

midi_hz now clamps its input as well. On a real-time thread a value that is merely wrong and a value that hangs are different severities.

Fixing the aliasing fixes *this* bug. Clamping the input means the next bad index produces a wrong note instead of silence — which is recoverable, and diagnosable. The same clamp appears in the ABC player's `midi_to_hz`, with the same reasoning.

A font built inside drawRect:

Asking for a font on every string, thirty times a second, eventually returned nil, and a nil value in an attributes dictionary raises — surfacing as a trap deep inside AppKit with a stack that mentions nothing about fonts.

A resource acquired in a draw loop and never released. It works, then works, and eventually the request fails — and the consequence surfaces somewhere else entirely, as a trap inside AppKit's attributed-string machinery with nothing in the stack about fonts.

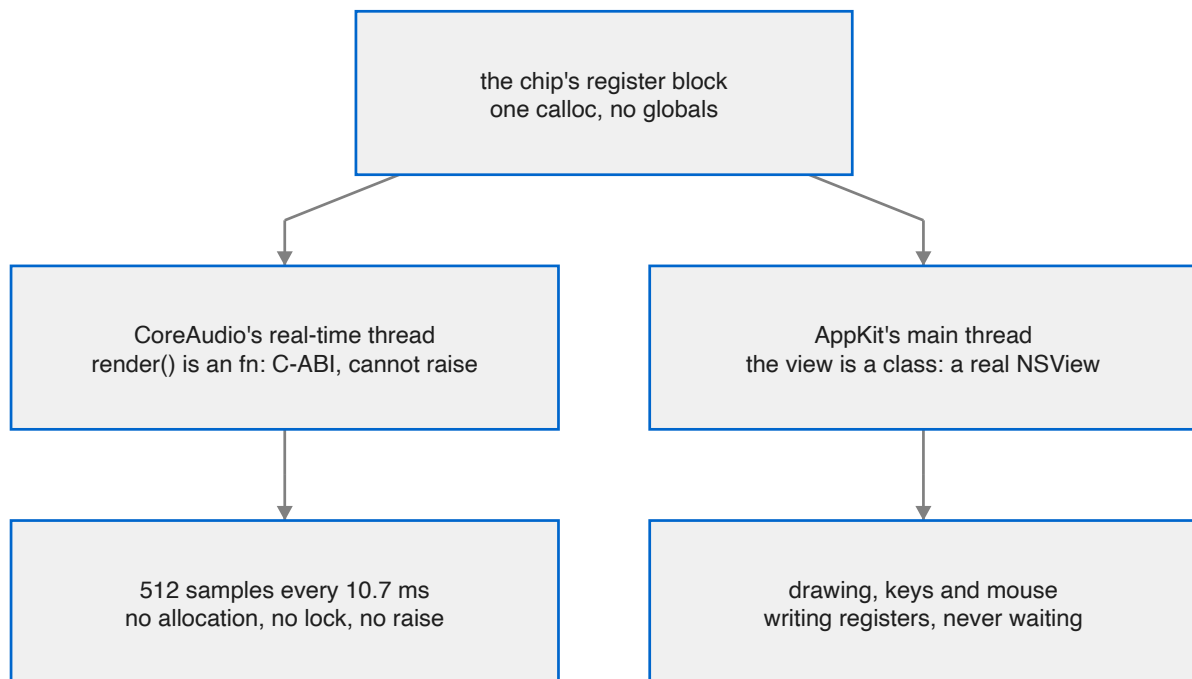
The fonts are now made once and retained. The ABC player does the same, inheriting the fix.

Why "found by measuring rather than by reading"

Both bugs are invisible to review. The `let` looks like an ordinary local assignment — it *is* an ordinary local assignment, in every other language. The font call looks like ordinary drawing code.

And both produce symptoms that point away from their cause: silence attributed to the audio system, an AppKit trap attributed to string drawing. What found them was a counter that stopped advancing, and then printing the offending value **from the thread that could safely print it** — because printing from the audio thread would have been its own deadline violation.

That last clause is a real technique. On a real-time thread you cannot log. You can write a value to a slot and let another thread read it, which is what the register block makes trivial.



8. What to understand

Nine things about this synthesiser are not obvious from listening to it.

1. The player routine is where the music is

The oscillator is the part everyone describes and the part that matters least. Three voices and no memory means everything characteristic happens in a routine that rewrites registers fifty times a second:

- a **chord** is one voice switching between three notes on consecutive frames
- a **held note** is kept alive by sweeping its pulse width
- a **drum** is noise plus a downward pitch sweep

So the interesting code is not the oscillator. It is this: fifty edits a second to a handful of registers, and out comes music.

2. The envelope table is the single most important detail

Decay and release are **not exponential**. The chip counts down at a rate divided further as the level falls — below 93, then 54, 26, 14 and 6 — so the tail flattens in five steps.

Replacing that with a smooth exponential is the single change that makes this sound like a synthesiser instead of a games machine.

Get every oscillator right and this wrong, and it sounds like a modern synth playing chip waveforms.

3. Selecting two waveforms ANDs them

Not a mix, not an average — a bitwise AND, because that is how the outputs were wired. Most of the timbres people remember come from an accident.

The interface follows the hardware: the ABC player's voice editor presents the four waveforms as **toggles rather than a radio group**, because presenting them as exclusive would both remove capability and teach the wrong model.

4. Noise has a pitch

The LFSR shifts on a rising edge of accumulator bit 19, so the frequency register controls the noise's character — low is a rumble, high is a hiss. A rising noise sweep is a **frequency** sweep, not a filter sweep.

This is why a chip drum is written as a note, and why the drum code sweeps `hz`.

5. Ring modulation is one XOR

A triangle wave folds according to its top bit. Ring modulation replaces that bit with the previous voice's:

```
if ((acc24 ^ ring_source_msb) & 0x800000) != 0:
    folded = (~acc24) & 0xFFFFFF
```

On a modular synthesiser this is a four-quadrant multiplier. Here it is one XOR in a fold test — *"and it is why bells and gongs sound the way they do."*

6. The filter is only conditionally stable, and the bug proved it

A Chamberlin state-variable filter is stable while $f + q < 2$. The original code clamped f to a flat 1.4, which does not express a constraint coupling two variables — and at the demo settings it sat at **f 1.099 against a bound of 1.116**. Inside by 1.5%, with nobody aware there was a bound.

Nothing found this until a tune began sweeping the cutoff, because nothing had ever changed it while a note was sounding.

Static settings never explore the parameter space. It took the ABC player's `[I:chip cutoff=...]` directive to walk it out.

7. NaN is permanent, and clamping the output cannot save you

The filter's state feeds back into itself, so one NaN poisons every subsequent sample. The synth goes silent for good.

The output clamp below cannot help, because by then the damage is in the state rather than the sample.

The general rule: in a feedback system, validate what goes **back in**, not what comes out. And `low != low` is the NaN test — the only value not equal to itself — which avoids an import an `fn` could not make anyway.

8. A loop whose trip count grows is a real-time bug

```
# Subtracting 2*pi until the argument lands in range is harmless for a
# filter coefficient and quietly ruinous for vibrato: that argument grows
# with the frame counter, so the loop gets one iteration longer every fifty
# frames -- on the audio thread, where the cost eventually shows up as a
```

```
# dropout and never as an error.
```

The vibrato calls `_sin(frame * 0.55)` and `frame` counts up forever. A loop-based range reduction gets slower for as long as the program runs, and the symptom arrives ten minutes in as a click with nothing to point at.

On a real-time thread, "correct but unboundedly slow" is a bug.

9. Silence is a worse failure than a wrong note

Two separate places clamp their input for the same reason:

```
""Clamped, because this runs on the audio thread and the octave loop below
is a loop: a nonsense note would be a hang, not a wrong pitch.""
```

On a real-time thread a value that is merely wrong and a value that hangs are different severities.

A wrong note is audible, diagnosable and recoverable. A hang on the audio thread is silence for the session, with no crash and no stack trace — and in the bug that produced this rule, it happened about three runs in five, *"which is exactly the failure rate that makes you blame the audio system."*

A short list of things that will bite

If you change...	...this happens
the envelope to a smooth exponential	it stops sounding like a chip
the phase accumulator to a float	the pitch quantisation and hard edges go
noise to a random generator	it hisses instead of rasping, and loses its pitch
<code>var prev</code> to <code>let prev</code> in the oscillator	hard sync and the noise clock stop, silently
<code>var left</code> to <code>let left</code> in the player	the drum's pitch sweep goes off by one frame
the pulse sweep to reach 0 or 4095	the voice drops out at the edge of every cycle
the envelope multiply before the DC centring	a thump on every note-on
<code>f</code> 's clamp back to a constant	the filter diverges at high cutoff with resonance
the NaN reset out of the filter	one divergence silences the synth until restart
<code>_sin</code> 's range reduction to a loop	a dropout ten minutes in, with no error
a <code>def</code> anywhere on the render path	it will not install; <code>fn</code> is the C-ABI kind

Running it

```
cocoamojo --build examples/chip/main.mojo -I examples/chip -o /tmp/chip
```

/tmp/chip	the built-in tune
/tmp/chip examples/chip/tunes/ode.abc	an ABC tune

Key	
Space	pause
1 2 3	mute a voice
< >	cutoff
- +	resonance
F	filter mode
Q	quit

The same engine, driven from a sample-accurate schedule instead of a 50 Hz score, is the [ABC player](#).